

These questions cover everything from ERDs to views and triggers. Work through them independently and bring your answers back. Answers will be discussed after the break.

Q1 Entity–Relationship Design (*ERDs*)

A music streaming service tracks **Users**, **Songs**, and **Playlists**.

- Each User has a unique `user_id`, a `username`, and an `email`.
- Each Song has a unique `song_id`, a `title`, and an `artist`.
- A Playlist has a unique `playlist_id`, a `name`, and belongs to **exactly one** User.
- A Playlist contains **many** Songs; a Song may appear in **many** Playlists.
- The relationship between Playlist and Song has an attribute: `position` (the track order).

(a) Draw the Chen ERD. Show entity rectangles, key attributes (underlined), regular attributes, relationship diamonds, and cardinalities.

Sketch your ERD here.

(b) Where does the `position` attribute live in your ERD, and why?

Q2 DDL – Translate the ERD to SQL

Write `CREATE TABLE` statements for all three entities from Q1 plus the M:N relationship. Use appropriate types and constraints; add a foreign key for the Playlist–User relationship with `ON DELETE CASCADE`, and an appropriate action for the Song–Playlist link. Include inline comments explaining each choice.

Use the schema below for all DML and RA questions.

Users(uid, username, email) Songs(sid, title, artist) Playlists(pid, name, uid) Contains(pid, sid, position)

Users			Songs			Playlists			Contains		
uid	username	email	sid	title	artist	pid	name	uid	pid	sid	pos
1	anya	anya@x.ca	10	Blinding Lights	The Weeknd	100	Chill	1	100	10	1
2	boris	boris@x.ca	11	Levitating	Dua Lipa	101	Hype	1	100	11	2
3	carmen	car@x.ca	12	Peaches	J. Bieber	102	Focus	2	101	10	1
			13	Stay	Kid LAROI				101	13	2
									102	11	1

Q3 DML – Queries

(a) The titles of all songs that appear in **at least two** different playlists. Use `GROUP BY` and `HAVING`.

(b) The usernames of users who have **no** playlists. Use a `LEFT JOIN` and `IS NULL`.

(c) For each user, the **total number of songs** across all their playlists (counting duplicates across playlists). Return `username` and the count.

Q4 Relational Algebra

(a) Using σ , π , and $\bowtie$: the titles of songs in the playlist named 'Chill'.

(b) Using $\mathcal{G}$: for each `pid`, the number of songs it contains. Write the RA expression, then the equivalent SQL.

(c) Using set difference: the `titles` of songs that have **never** been added to any playlist.

Q5 Views & Triggers

(a) Create a view `PopularSongs` containing the `sid` and `title` of every song that appears in more than one playlist. Write the `CREATE VIEW` statement.

(b) Write a trigger `PreventDuplicatePosition` that fires **before** an insert on `Contains` and raises an error if the same (`pid`, `position`) pair already exists. (*Sketch the structure, exact error-raising syntax varies by DBMS.*)

(c) Should this trigger use `FOR EACH ROW` or `FOR EACH STATEMENT`? Why?
.....